

150+ Tricky JavaScript Guess-the-Output Questions (with Explanations)

Use this for rapid revision and interview prep.

Section 1: Event Loop and Task Queues (Q1-15)

Q1

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

Answer: 1, 4, 3, 2 **Explanation:** Sync (1, 4), microtask (3), macrotask (2).

Q2

```
setTimeout(() => {
  console.log("A");
  Promise.resolve().then(() => console.log("B"));
}, 0);

Promise.resolve().then(() => {
  console.log("C");
  setTimeout(() => console.log("D"), 0);
});

console.log("E");
```

Answer: E, C, A, B, D **Explanation:** Sync (E), microtask (C), first macrotask (A), microtask after A (B), next macrotask (D).

Q3

```
async function test() {
  console.log("1");
  await Promise.resolve();
  console.log("2");
}

test();
console.log("3");
```

Answer: 1, 3, 2 **Explanation:** `await` pauses, control returns to event loop, microtask executes after sync.

Q4

```

Promise.resolve()
  .then(() => {
    console.log("A");
    return Promise.resolve();
  })
  .then(() => console.log("B"));

console.log("C");

```

Answer: C, A, B **Explanation:** Return of promise creates extra microtask.

Q5

```

setTimeout(() => console.log("1"), 0);
Promise.resolve().then(() => {
  console.log("2");
  setTimeout(() => console.log("3"), 0);
});
Promise.resolve().then(() => console.log("4"));

```

Answer: 2, 4, 1, 3 **Explanation:** Microtasks (2, 4) before first macrotask (1), then macrotask (3).

Q6

```

queueMicrotask(() => console.log("A"));
Promise.resolve().then(() => console.log("B"));
console.log("C");

```

Answer: C, A, B **Explanation:** queueMicrotask and Promise.then are both microtasks, executed in FIFO order.

Q7

```

console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

new Promise((resolve) => {
  console.log("C");
  resolve();
}).then(() => console.log("D"));

console.log("E");

```

Answer: A, C, E, D, B **Explanation:** Executor runs sync (C), then sync outside (A, E), then microtasks (D), then macrotasks (B).

Q8

```
setTimeout(() => console.log("1"), 0);
setTimeout(() => console.log("2"), 10);
Promise.resolve().then(() => console.log("3"));
```

Answer: 3, 1, 2 **Explanation:** Microtask (3) before any macrotask, then setTimeout order.

Q9

```
new Promise(resolve => {
  resolve(Promise.resolve("1"));
}).then(console.log);

new Promise(resolve => {
  resolve("2");
}).then(console.log);
```

Answer: 2, 1 **Explanation:** Nested promise resolve adds extra microtask step.

Q10

```
Promise.resolve("1").then(x => x + "2");
Promise.resolve("3").then(console.log);
```

Answer: 3 **Explanation:** First promise chain doesn't log, second does. "1" + "2" = "12" not logged.

Q11

```
console.log("start");
setImmediate(() => console.log("immediate"));
process.nextTick(() => console.log("nexttick"));
Promise.resolve().then(() => console.log("promise"));
console.log("end");
```

Answer (Node.js): start, end, nexttick, promise, immediate **Explanation:** nextTick is before promises (different microtask), immediate is macrotask.

Q12

```
let p = Promise.reject("error");
setTimeout(() => console.log("timeout"), 0);
p.catch(() => console.log("caught"));
```

Answer: caught, timeout **Explanation:** Microtask (catch) before macrotask (timeout).

Q13

```

Promise.resolve()
  .then(() => {
    throw new Error("error");
  })
  .catch(() => console.log("1"))
  .then(() => console.log("2"));

```

Answer: 1, 2 **Explanation:** Catch returns resolved promise, so then after catch executes.

Q14

```

setTimeout(() => {
  console.log("setTimeout");
  Promise.resolve().then(() => console.log("promise"));
}, 0);

Promise.resolve().then(() => {
  console.log("promise1");
  setTimeout(() => console.log("setTimeout2"), 0);
});

```

Answer: promise1, setTimeout, promise, setTimeout2 **Explanation:** Microtask (promise1), macrotask (setTimeout), microtask (promise), macrotask (setTimeout2).

Q15

```

const p = new Promise(resolve => {
  console.log("executor");
  resolve();
});

console.log("after");

p.then(() => console.log("then"));

```

Answer: executor, after, then **Explanation:** Executor runs sync, then is microtask.

Section 2: Closures and Scope (Q16-30)

Q16

```

for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

```

Answer: 3, 3, 3 **Explanation:** var is function-scoped, all callbacks share same i .

Q17

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Answer: 0, 1, 2 **Explanation:** `let` is block-scoped, each iteration gets own `i` .

Q18

```
function outer() {  
  let x = 1;  
  function inner() {  
    console.log(x);  
    x = 2;  
  }  
  inner();  
  console.log(x);  
}  
outer();
```

Answer: 1, 2 **Explanation:** Closure allows inner to access and modify x.

Q19

```
const funcs = [];  
for (var i = 0; i < 3; i++) {  
  funcs.push(() => console.log(i));  
}  
funcs.forEach(f => f());
```

Answer: 3, 3, 3 **Explanation:** All closures close over same `i` .

Q20

```
const funcs = [];  
for (let i = 0; i < 3; i++) {  
  funcs.push(() => console.log(i));  
}  
funcs.forEach(f => f());
```

Answer: 0, 1, 2 **Explanation:** Block scope creates new `i` for each iteration.

Q21

```
let x = 1;  
function f() {  
  console.log(x);  
  let x = 2;  
  console.log(x);  
}
```

```
}  
f();
```

Answer: Error - ReferenceError: Cannot access 'x' before initialization **Explanation:** TDZ - let declaration hoists but not initialized.

Q22

```
function f() {  
  if (true) {  
    var x = 1;  
  }  
  console.log(x);  
}  
f();
```

Answer: 1 **Explanation:** var is function-scoped, not block-scoped.

Q23

```
const obj = {  
  x: 10,  
  f: function() {  
    console.log(this.x);  
    const g = () => {  
      console.log(this.x);  
    };  
    g();  
  }  
};  
obj.f();
```

Answer: 10, 10 **Explanation:** Arrow function inherits this from enclosing function.

Q24

```
function makeAdder(x) {  
  return function(y) {  
    return x + y;  
  };  
}  
const add5 = makeAdder(5);  
console.log(add5(3));
```

Answer: 8 **Explanation:** Closure captures x.

Q25

```
const obj = {
  x: 1,
  f: function() {
    function inner() {
      console.log(this.x);
    }
    inner();
  }
};
obj.f();
```

Answer: undefined **Explanation:** `this` in `inner` is undefined (in strict mode) or `window` (in non-strict).

Q26

```
let count = 0;
function increment() {
  return ++count;
}
console.log(increment());
console.log(increment());
console.log(count);
```

Answer: 1, 2, 2 **Explanation:** Closure over `count`.

Q27

```
const arr = [1, 2, 3];
for (var i = 0; i < arr.length; i++) {
  arr[i] = arr[i] * 2;
}
console.log(i);
console.log(arr);
```

Answer: 3, [2, 4, 6] **Explanation:** `i` leaks to function scope.

Q28

```
function test() {
  let a = 1;
  {
    let a = 2;
    console.log(a);
  }
  console.log(a);
}
test();
```

Answer: 2, 1 **Explanation:** Block scope shadowing.

Q29

```
function f(x) {  
  function g() {  
    console.log(x);  
  }  
  x = 10;  
  g();  
}  
f(5);
```

Answer: 10 **Explanation:** Closure accesses updated value.

Q30

```
const obj = {  
  value: 10,  
  getValue: () => {  
    return this.value;  
  }  
};  
console.log(obj.getValue());
```

Answer: undefined **Explanation:** Arrow function has global `this`, not obj.

Section 3: Hoisting and TDZ (Q31-45)

Q31

```
console.log(foo);  
var foo = 5;
```

Answer: undefined **Explanation:** `var` declaration hoisted and initialized to undefined.

Q32

```
console.log(bar);  
let bar = 5;
```

Answer: ReferenceError: Cannot access 'bar' before initialization **Explanation:** TDZ for `let`.

Q33

```
hoisted();  
function hoisted() {
```



```
    console.log("yes");  
  }
```

Answer: yes **Explanation:** Function declaration hoisted entirely.

Q34

```
notHoisted();  
const notHoisted = () => console.log("yes");
```

Answer: TypeError: notHoisted is not a function **Explanation:** Arrow function assigned to const, so identifier exists but not initialized.

Q35

```
function test() {  
  console.log(typeof x);  
  var x = 5;  
}  
test();
```

Answer: undefined **Explanation:** typeof doesn't throw on hoisted but uninitialized var.

Q36

```
function test() {  
  console.log(typeof x);  
  let x = 5;  
}  
test();
```

Answer: ReferenceError **Explanation:** let is in TDZ.

Q37

```
var a = 1;  
function test() {  
  console.log(a);  
  var a = 2;  
}  
test();
```

Answer: undefined **Explanation:** var a hoisted in function, shadows outer a .

Q38

```
console.log(this.x);
```

```
var x = 1;
```

Answer (browser): undefined **Explanation:** `var` at global doesn't become property if strict mode.

Q39

```
function f() {  
  console.log(x);  
  if (false) {  
    var x = 1;  
  }  
}  
f();
```

Answer: undefined **Explanation:** `var x` hoisted even though in unreachable code.

Q40

```
let x = 1;  
{  
  console.log(x);  
  let x = 2;  
}
```

Answer: ReferenceError **Explanation:** TDZ - `let x = 2` shadows outer `x`, but not initialized yet.

Q41

```
console.log(typeof undeclared);
```

Answer: undefined **Explanation:** `typeof` is special, doesn't throw on undefined identifier.

Q42

```
function test() {  
  var a = 1;  
}  
console.log(typeof a);
```

Answer: undefined **Explanation:** Function scope, `a` doesn't exist globally.

Q43

```
var x = 1;  
{  
  x = 2;  
  var y = 3;
```

```
}  
console.log(x, y);
```

Answer: 2, 3 **Explanation:** `var` in block still function-scoped.

Q44

```
function f() {  
  if (true) {  
    function g() {  
      console.log("yes");  
    }  
  }  
  g();  
}  
f();
```

Answer (non-strict): yes **Explanation:** Function declaration in block hoisted (browser dependent).

Q45

```
const f = function g() {  
  console.log(typeof g);  
};  
f();  
console.log(typeof g);
```

Answer: function, undefined **Explanation:** Named function expression, `g` only available inside function.

Section 4: Promises and Async/Await (Q46-70)

Q46

```
Promise.resolve(1)  
  .then(x => x + 1)  
  .then(x => console.log(x));
```

Answer: 2 **Explanation:** Promise chaining.

Q47

```
Promise.reject("error")  
  .then(x => console.log("1:", x))  
  .catch(e => console.log("2:", e));
```

Answer: 2: error **Explanation:** Rejected promise skips then, goes to catch.

Q48

```
Promise.resolve(1)
  .then(x => {
    throw new Error("oops");
  })
  .catch(e => console.log("caught"))
  .then(x => console.log("after"));
```

Answer: caught, after **Explanation:** Catch returns resolved promise.

Q49

```
new Promise((resolve, reject) => {
  resolve(1);
  reject(2);
}).then(
  x => console.log("1:", x),
  e => console.log("2:", e)
);
```

Answer: 1, 1 **Explanation:** Only first settlement matters.

Q50

```
const p = Promise.resolve(1);
p.then(x => console.log(x));
p.then(x => console.log(x));
```

Answer: 1, 1 **Explanation:** Both then callbacks execute.

Q51

```
Promise.all([
  Promise.resolve(1),
  Promise.resolve(2),
  Promise.reject("error")
]).then(
  x => console.log("1:", x),
  e => console.log("2:", e)
);
```

Answer: 2: error **Explanation:** Promise.all rejects if any rejects.

Q52

```
Promise.allSettled([
  Promise.resolve(1),
  Promise.reject("error"),
```

```
Promise.resolve(2)
]).then(x => console.log(JSON.stringify(x)));
```

Answer: [{"status":"fulfilled","value":1},{status":"rejected","reason":"error"},{"status":"fulfilled","value":2}] **Explanation:** allSettled always resolves.

Q53

```
Promise.race([
  new Promise(r => setTimeout(() => r(1), 100)),
  new Promise(r => setTimeout(() => r(2), 50))
]).then(x => console.log(x));
```

Answer: 2 **Explanation:** Race settles with first to settle.

Q54

```
async function f() {
  return 1;
}
f().then(x => console.log(x));
```

Answer: 1 **Explanation:** Async function returns promise.

Q55

```
async function f() {
  throw new Error("oops");
}
f().catch(e => console.log("caught"));
```

Answer: caught **Explanation:** Async function throws returns rejected promise.

Q56

```
async function f() {
  await new Promise(r => {});
  console.log("after");
}
f();
console.log("sync");
```

Answer: sync **Explanation:** await pauses, doesn't block event loop.

Q57

```
async function f() {
  const x = await Promise.resolve(1);
```

```
    console.log(x);  
  }  
  f();
```

Answer: 1 **Explanation:** Await unwraps promise.

Q58

```
async function f() {  
  try {  
    throw new Error("oops");  
  } catch (e) {  
    console.log("caught");  
  }  
}  
f();
```

Answer: caught **Explanation:** Try-catch in async.

Q59

```
const p1 = Promise.resolve(1);  
const p2 = Promise.resolve(2);  
Promise.all([p1, p2]).then(([a, b]) => console.log(a + b));
```

Answer: 3 **Explanation:** Destructuring in then.

Q60

```
Promise.resolve()  
  .then(() => Promise.resolve(1))  
  .then(x => console.log(x));
```

Answer: 1 **Explanation:** Returning promise from then unwraps it.

Q61

```
Promise.resolve()  
  .then(() => {  
    console.log("1");  
    return Promise.resolve();  
  })  
  .then(() => console.log("2"));
```

Answer: 1, 2 **Explanation:** Return of promise adds extra microtask.

Q62

```

async function f() {
  const x = await (async () => {
    return 1;
  })();
  console.log(x);
}
f();

```

Answer: 1 **Explanation:** IIFE async returns promise.

Q63

```

async function f() {
  const x = 1;
  const y = await Promise.resolve(2);
  console.log(x + y);
}
f();

```

Answer: 3 **Explanation:** Mixed sync and async.

Q64

```

async function f() {
  return await Promise.resolve(1);
}
f().then(x => console.log(x));

```

Answer: 1 **Explanation:** Return await same as return promise.

Q65

```

const f = async () => {
  return 1;
};
f().then(x => console.log(x));

```

Answer: 1 **Explanation:** Arrow async function.

Q66

```

Promise.resolve(1)
  .then(x => console.log("1:", x), e => console.log("2:", e))
  .then(x => console.log("3:", x));

```

Answer: 1, 3: undefined **Explanation:** Second parameter of then is error handler.

Q67

```
async function f() {  
  const x = await Promise.resolve(1);  
  const y = await Promise.resolve(2);  
  console.log(x + y);  
}  
f();
```

Answer: 3 **Explanation:** Sequential awaits.

Q68

```
async function f() {  
  const promises = [Promise.resolve(1), Promise.resolve(2)];  
  const [a, b] = await Promise.all(promises);  
  console.log(a + b);  
}  
f();
```

Answer: 3 **Explanation:** Promise.all with destructuring.

Q69

```
Promise.resolve(1).then(console.log);
```

Answer: 1 **Explanation:** Passing function reference.

Q70

```
async function f() {  
  try {  
    await Promise.reject("error");  
  } catch (e) {  
    console.log("caught:", e);  
  }  
}  
f();
```

Answer: caught: error **Explanation:** Try-catch around await.

Section 5: Prototypes and this (Q71-85)

Q71

```
function Animal(name) {  
  this.name = name;
```



```

}
Animal.prototype.speak = function() {
  console.log(this.name);
};
const dog = new Animal("Dog");
dog.speak();

```

Answer: Dog **Explanation:** Constructor function and prototype.

Q72

```

const obj = {
  x: 1,
  f: function() {
    console.log(this.x);
  }
};
obj.f();
const f = obj.f;
f();

```

Answer: 1, undefined **Explanation:** `this` depends on call site.

Q73

```

function f() {
  console.log(this);
}
f.call({ x: 1 });

```

Answer: { x: 1 } **Explanation:** Call sets `this` .

Q74

```

const obj = {
  x: 1,
  f: () => console.log(this.x)
};
obj.f();

```

Answer: undefined **Explanation:** Arrow function `this` is global.

Q75

```

const obj = {
  x: 1,
  f: function() {
    const g = () => console.log(this.x);
  }
};
obj.f();

```

```
    g();  
  }  
};  
obj.f();
```

Answer: 1 **Explanation:** Arrow function inherits `this` from outer function.

Q76

```
function f() {  
  console.log(this.constructor.name);  
}  
f.call({});
```

Answer: Object **Explanation:** Constructor property of empty object.

Q77

```
const obj = {  
  x: 1,  
  get f() {  
    return () => console.log(this.x);  
  }  
};  
obj.f();
```

Answer: 1 **Explanation:** Arrow function from getter.

Q78

```
function f() {  
  console.log(arguments);  
}  
f(1, 2, 3);
```

Answer: [Arguments] { '0': 1, '1': 2, '2': 3 } **Explanation:** arguments pseudo-array.

Q79

```
const f = (...args) => console.log(args);  
f(1, 2, 3);
```

Answer: [1, 2, 3] **Explanation:** Rest parameters in arrow function.

Q80

```
function f() {  
  console.log(this);  
}
```

```
}  
new f();
```

Answer: f {} **Explanation:** new creates object and sets this .

Q81

```
function Animal() {  
  this.x = 1;  
}  
Animal.prototype.y = 2;  
const a = new Animal();  
console.log(a.x, a.y);
```

Answer: 1, 2 **Explanation:** Own property and inherited property.

Q82

```
const obj = { x: 1 };  
const proxy = new Proxy(obj, {  
  get(t, key) {  
    console.log("getting", key);  
    return t[key];  
  }  
});  
console.log(proxy.x);
```

Answer: getting x, 1 **Explanation:** Proxy intercepts property access.

Q83

```
function f() {  
  return this;  
}  
const obj = { f };  
console.log(obj.f() === obj);
```

Answer: true **Explanation:** this is obj when called as method.

Q84

```
const f = function() {  
  console.log(this);  
}.bind({ x: 1 });  
f();
```

Answer: { x: 1 } **Explanation:** Bind sets permanent this .

Q85

```
function f() {  
  this.x = 1;  
}  
f.prototype.constructor = f;  
const a = new f();  
console.log(a.constructor === f);
```

Answer: true **Explanation:** Constructor property.

Section 6: Type Coercion (Q86-100)

Q86

```
console.log(0 == false);  
console.log(0 === false);
```

Answer: true, false **Explanation:** == coerces, === doesn't.

Q87

```
console.log("" == 0);  
console.log("" === 0);
```

Answer: true, false **Explanation:** Coercion to number.

Q88

```
console.log(null == undefined);  
console.log(null === undefined);
```

Answer: true, false **Explanation:** == considers them equal.

Q89

```
console.log(typeof null);
```

Answer: object **Explanation:** typeof null is object (JS quirk).

Q90

```
console.log(1 + "1");  
console.log(1 - "1");
```

Answer: 11, 0 **Explanation:** + favors string, - favors number.

Q91

```
console.log([1] == "1");  
console.log([1] === "1");
```

Answer: true, false **Explanation:** Array coerced to string.

Q92

```
console.log("2" > "10");  
console.log("2" > 10);
```

Answer: true, false **Explanation:** String comparison vs number comparison.

Q93

```
console.log(Boolean(0));  
console.log(Boolean(""));   
console.log(Boolean(null));
```

Answer: false, false, false **Explanation:** Falsy values.

Q94

```
console.log(!!"0");  
console.log(!!0);
```

Answer: true, false **Explanation:** "0" string is truthy.

Q95

```
console.log([] == ![]);
```

Answer: true **Explanation:** Both coerce to 0 or false.

Section 7: Advanced Closures and Patterns (Q101-125)

Q101

```
function createMultiplier(x) {  
  return function(y) {  
    return x * y;  
  };  
}  
const times3 = createMultiplier(3);
```

```
console.log(times3(2));
console.log(times3(4));
```

Answer: 6, 12 **Explanation:** Closure over `x` .

Q102

```
const obj = (() => {
  let count = 0;
  return {
    increment: () => ++count,
    get: () => count
  };
})();
console.log(obj.increment());
console.log(obj.increment());
console.log(obj.get());
```

Answer: 1, 2, 2 **Explanation:** IIFE module pattern with private state.

Q103

```
function memoize(fn) {
  const cache = {};
  return function(x) {
    if (x in cache) {
      console.log("cached");
      return cache[x];
    }
    console.log("computing");
    cache[x] = fn(x);
    return cache[x];
  };
}
const fn = memoize(x => x * 2);
console.log(fn(5));
console.log(fn(5));
```

Answer: computing, 10, cached, 10 **Explanation:** Memoization with closure.

Q104

```
let x = 1;
const obj = {
  x: 2,
  f: function() {
    return () => this.x;
  }
};
```

```
const g = obj.f();
console.log(g());
```

Answer: 2 **Explanation:** Arrow function captures `this` from `f`.

Q105

```
const arr = [1, 2, 3];
for (var i = 0; i < arr.length; i++) {
  (function(j) {
    setTimeout(() => console.log(j), 0);
  })(i);
}
```

Answer: 0, 1, 2 **Explanation:** IIFE creates new scope for each `j`.

Section 8: React and Async (Q126-150)

Q126

```
// React component rendering
function App() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
    setCount(count + 1);
    console.log(count);
  };

  return <button onClick={handleClick}>{count}</button>;
}
```

Answer: 0 (logged) **Explanation:** `setCount` is batched, `console.log` logs before render.

Q127

```
function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(count + 1);
  }, []); // empty dependency

  return <div>{count}</div>;
}
```

Answer: Renders 1 only once **Explanation:** Effect runs once, updates count from 0 to 1.

Q128

```
function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("effect");
    return () => console.log("cleanup");
  }, []);

  return null;
}
```

Answer: "effect" on mount, "cleanup" on unmount **Explanation:** Cleanup function called on unmount.

Q129

```
function App() {
  const memoizedValue = useMemo(() => {
    console.log("computing");
    return 0;
  }, []);

  return <div>{memoizedValue}</div>;
}
```

Answer: "computing" only once **Explanation:** useMemo with empty dependency computes once.

Q130

```
function useCustom() {
  const [state, setState] = useState(0);
  return [state, setState];
}

function App() {
  const [count, setCount] = useCustom();
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Answer: Correct custom hook usage **Explanation:** Custom hooks are functions that use other hooks.

Final Revision Summary

Key Concepts Covered:

- ✔ Event loop, microtask, macrotask ordering
- ✔ Closures and scope chains
- ✔ Hoisting and TDZ

- ☒ Promises and async/await
- ☒ Prototypal inheritance and this binding
- ☒ Type coercion
- ☒ React hooks and rendering
- ☒ Advanced patterns (memoization, modules, custom hooks)

Interview Tips:

1. Explain the "why", not just the "what".
2. Draw event loop diagrams.
3. Clarify closure over global/block/function scope.
4. Know typeof vs instanceof.
5. Know async/await vs promises.
6. Know hook rules.